Object Oriented Programming

CS22301

UNIT IV

COLLECTIONS, MULTITHREADING AND GENERIC PROGRAMMING



A. Bamila Virgin Louis

Asso.Prof/ CSE

SXCCE

UNIT IV

The Collections Framework: Collections Overview - The Collection Interface: The List Interface - The Set Interface- The Collection Classes: The ArrayList Class - The LinkedList Class - Accessing a Collection via an Iterator – The For-Each Alternative to Iterators. Multithreading: Differences between multi-threading and multitasking, thread life cycle, creating threads, synchronizing threads, Inter-thread communication, daemon threads, thread groups. Generic Programming – Generic classes – generic methods

COLLECTIONS

THE COLLECTIONS FRAMEWORK

- This package **java.util** contains a large assortment of classes and interfaces that support a broad range of functionality.
- **java.util** has classes that
  - generate pseudorandom numbers,
  - manage date and time,
  - observe events,
  - manipulate sets of bits, tokenize strings, and
  - handle formatted data.

List of classes in java.util

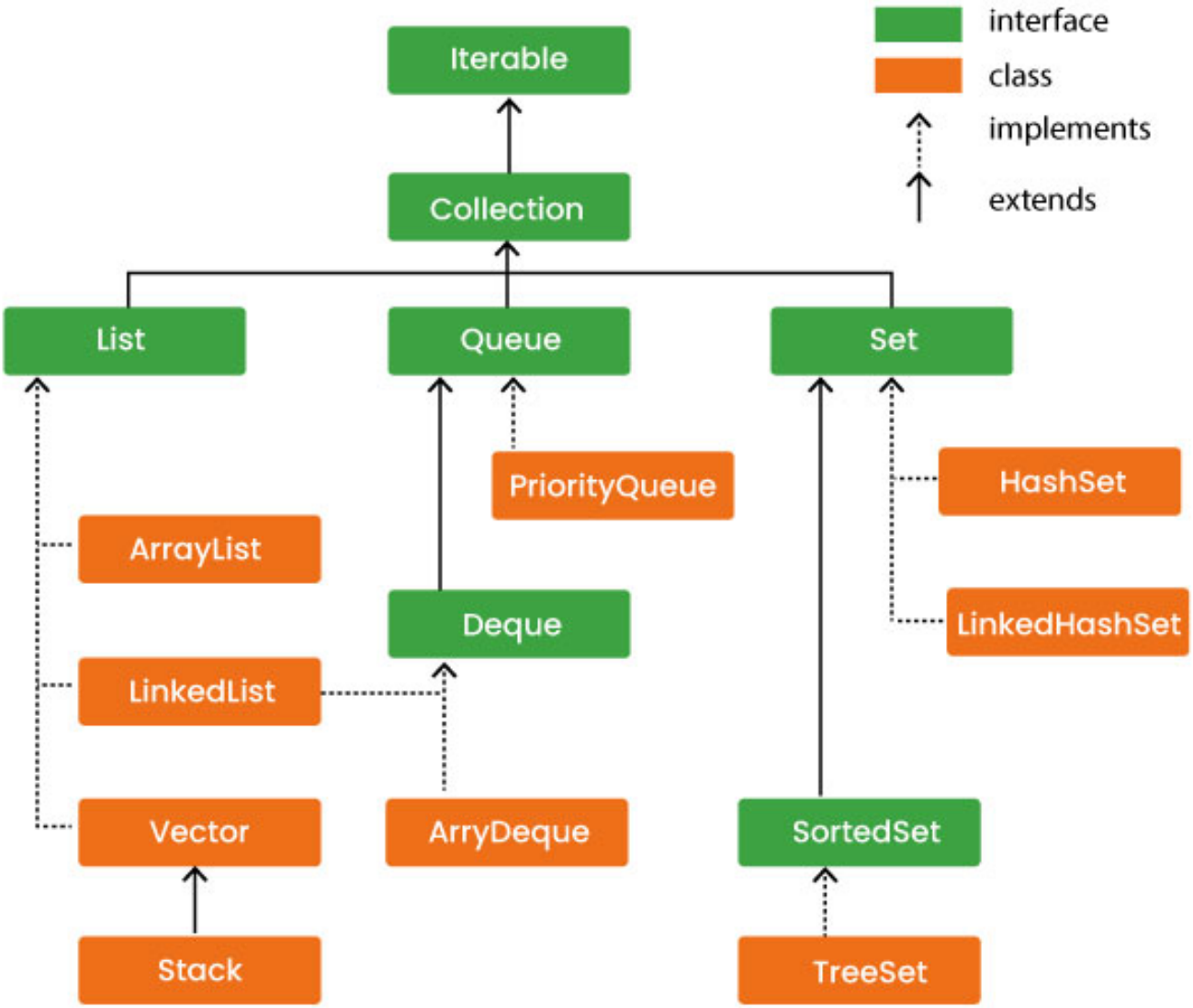
AbstractCollection	EventObject	Random
AbstractList	FormattableFlags	ResourceBundle
AbstractMap	Formatter	Scanner
AbstractQueue	GregorianCalendar	ServiceLoader (Added by Java SE 6.)
AbstractSequentialList	HashMap	SimpleTimeZone
AbstractSet	HashSet	Stack
ArrayDeque (Added by Java SE 6.)	Hashtable	StringTokenizer
ArrayList	IdentityHashMap	Timer
Arrays	LinkedHashMap	TimerTask
BitSet	LinkedHashSet	TimeZone
Calendar	LinkedList	TreeMap
Collections	ListResourceBundle	TreeSet
Currency	Locale	UUID
Date	Observable	Vector
Dictionary	PriorityQueue	WeakHashMap
EnumMap	Properties	
EnumSet	PropertyPermission	
EventListenerProxy	PropertyResourceBundle	

Collections Overview

- The Java Collections Framework standardizes the way in which groups of objects are handled by your programs.
- Goals

1) the framework had to be high-performance. The implementations for the fundamental collections (dynamic arrays, linked lists, trees, and hash tables) are highly efficient.
2) the framework had to allow different types of collections to work in a similar manner and with a high degree of interoperability.
3) extending and/or adapting a collection had to be easy.

THE COLLECTION FRAMEWORK



THE COLLECTION INTERFACES

- **Collection** is a generic interface that has this declaration:

interface Collection<E>

Interface	Description
Collection	Enables you to work with groups of objects; it is at the top of the collections hierarchy.
Deque	Extends **Queue** to handle a double-ended queue. (Added by Java SE 6.)
List	Extends **Collection** to handle sequences (lists of objects).
NavigableSet	Extends **SortedSet** to handle retrieval of elements based on closest-match searches. (Added by Java SE 6.)
Queue	Extends **Collection** to handle special types of lists in which elements are removed only from the head.
Set	Extends **Collection** to handle sets, which must contain unique elements.
SortedSet	Extends **Set** to handle sorted sets.

Method	Description
boolean add(E *obj*)	Adds *obj* to the invoking collection. Returns **true** if *obj* was added to the collection. Returns **false** if *obj* is already a member of the collection and the collection does not allow duplicates.
boolean addAll(Collection<? extends E> *c*)	Adds all the elements of *c* to the invoking collection. Returns **true** if the operation succeeded (i.e., the elements were added). Otherwise, returns **false**.
void clear( )	Removes all elements from the invoking collection.
boolean contains(Object *obj*)	Returns **true** if *obj* is an element of the invoking collection. Otherwise, returns **false**.
boolean containsAll(Collection<?> *c*)	Returns **true** if the invoking collection contains all elements of *c*. Otherwise, returns **false**.
boolean equals(Object *obj*)	Returns **true** if the invoking collection and *obj* are equal. Otherwise, returns **false**.
int hashCode( )	Returns the hash code for the invoking collection.
boolean isEmpty( )	Returns **true** if the invoking collection is empty. Otherwise, returns **false**.

Iterator<E> iterator( )	Returns an iterator for the invoking collection.
boolean remove(Object *obj*)	Removes one instance of *obj* from the invoking collection. Returns **true** if the element was removed. Otherwise, returns **false**.
boolean removeAll(Collection<?> *c*)	Removes all elements of *c* from the invoking collection. Returns **true** if the collection changed (i.e., elements were removed). Otherwise, returns **false**.
boolean retainAll(Collection<?> *c*)	Removes all elements from the invoking collection except those in *c*. Returns **true** if the collection changed (i.e., elements were removed). Otherwise, returns **false**.
int size( )	Returns the number of elements held in the invoking collection.
Object[ ] toArray( )	Returns an array that contains all the elements stored in the invoking collection. The array elements are copies of the collection elements.

The List Interface

Method	Description
void add(int *index*, E *obj*)	Inserts *obj* into the invoking list at the index passed in *index*. Any preexisting elements at or beyond the point of insertion are shifted up. Thus, no elements are overwritten.
boolean addAll(int *index*, Collection<? extends E> *c*)	Inserts all elements of *c* into the invoking list at the index passed in *index*. Any preexisting elements at or beyond the point of insertion are shifted up. Thus, no elements are overwritten. Returns **true** if the invoking list changes and returns **false** otherwise.
E get(int *index*)	Returns the object stored at the specified index within the invoking collection.
int indexOf(Object *obj*)	Returns the index of the first instance of *obj* in the invoking list. If *obj* is not an element of the list, –1 is returned.
int lastIndexOf(Object *obj*)	Returns the index of the last instance of *obj* in the invoking list. If *obj* is not an element of the list, –1 is returned.

The List Interface

ListIterator<E> listIterator( )	Returns an iterator to the start of the invoking list.
ListIterator<E> listIterator(int *index*)	Returns an iterator to the invoking list that begins at the specified index.
E remove(int *index*)	Removes the element at position *index* from the invoking list and returns the deleted element. The resulting list is compacted. That is, the indexes of subsequent elements are decremented by one.
E set(int *index*, E *obj*)	Assigns *obj* to the location specified by *index* within the invoking list.
List<E> subList(int *start*, int *end*)	Returns a list that includes elements from *start* to *end*–1 in the invoking list. Elements in the returned list are also referenced by the invoking object.

The Set Interface

Method	Description
Comparator<? super E> comparator( )	Returns the invoking sorted set's comparator. If the natural ordering is used for this set, **null** is returned.
E first( )	Returns the first element in the invoking sorted set.
SortedSet<E> headSet(E *end*)	Returns a **SortedSet** containing those elements less than *end* that are contained in the invoking sorted set. Elements in the returned sorted set are also referenced by the invoking sorted set.
E last( )	Returns the last element in the invoking sorted set.
SortedSet<E> subSet(E *start*, E *end*)	Returns a **SortedSet** that includes those elements between *start* and *end*–1. Elements in the returned collection are also referenced by the invoking object.
SortedSet<E> tailSet(E *start*)	Returns a **SortedSet** that contains those elements greater than or equal to *start* that are contained in the sorted set. Elements in the returned set are also referenced by the invoking object.

The NavigableSet Interface

Method	Description
E ceiling(E *obj*)	Searches the set for the smallest element *e* such that *e* >= *obj*. If such an element is found, it is returned. Otherwise, **null** is returned.
Iterator<E> descendingIterator( )	Returns an iterator that moves from the greatest to least. In other words, it returns a reverse iterator.
NavigableSet<E> descendingSet( )	Returns a **NavigableSet** that is the reverse of the invoking set. The resulting set is backed by the invoking set.
E floor(E *obj*)	Searches the set for the largest element *e* such that *e* <= *obj*. If such an element is found, it is returned. Otherwise, **null** is returned.
NavigableSet<E> headSet(E *upperBound*, boolean *incl*)	Returns a **NavigableSet** that includes all elements from the invoking set that are less than *upperBound*. If *incl* is **true**, then an element equal to *upperBound* is included. The resulting set is backed by the invoking set.
E higher(E *obj*)	Searches the set for the largest element *e* such that *e* > *obj*. If such an element is found, it is returned. Otherwise, **null** is returned.
E lower(E *obj*)	Searches the set for the lowest element *e* such that *e* < *obj*. If such an element is found, it is returned. Otherwise, **null** is returned.

The NavigableSet Interface

E pollFirst( )	Returns the first element, removing the element in the process. Because the set is sorted, this is the element with the least value. **null** is returned if the set is empty.
E pollLast( )	Returns the last element, removing the element in the process. Because the set is sorted, this is the element with the greatest value. **null** is returned if the set is empty.
NavigableSet<E> subSet(E *lowerBound*, boolean *lowIncl*, E *upperBound*, boolean *highIncl*)	Returns a **NavigableSet** that includes all elements from the invoking set that are greater than *lowerBound* and less than *upperBound*. If *lowIncl* is **true**, then an element equal to *lowerBound* is included. If *highIncl* is **true**, then an element equal to *upperBound* is included. The resulting set is backed by the invoking set.
NavigableSet<E> tailSet(E *lowerBound*, boolean *incl*)	Returns a **NavigableSet** that includes all elements from the invoking set that are greater than *lowerBound*. If *incl* is **true**, then an element equal to **lowerBound** is included. The resulting set is backed by the invoking set.

THE COLLECTION CLASSES

Class	Description
AbstractCollection	Implements most of the **Collection** interface.
AbstractList	Extends **AbstractCollection** and implements most of the **List** interface.
AbstractQueue	Extends **AbstractCollection** and implements parts of the **Queue** interface.
AbstractSequentialList	Extends **AbstractList** for use by a collection that uses sequential rather than random access of its elements.
LinkedList	Implements a linked list by extending **AbstractSequentialList**.
ArrayList	Implements a dynamic array by extending **AbstractList**.
ArrayDeque	Implements a dynamic double-ended queue by extending **AbstractCollection** and implementing the **Deque** interface. (Added by Java SE 6.)
AbstractSet	Extends **AbstractCollection** and implements most of the **Set** interface.
EnumSet	Extends **AbstractSet** for use with **enum** elements.
HashSet	Extends **AbstractSet** for use with a hash table.
LinkedHashSet	Extends **HashSet** to allow insertion-order iterations.
PriorityQueue	Extends **AbstractQueue** to support a priority-based queue.
TreeSet	Implements a set stored in a tree. Extends **AbstractSet**.

The ArrayList Class

- **ArrayList** is a generic class that has this declaration:
- class ArrayList<E>

- **ArrayList** has the constructors shown here:

  ArrayList( )

  ArrayList(Collection<? extends E> *c*)

  ArrayList(int *capacity*)

The ArrayList Class

```
// Demonstrate ArrayList.
import java.util.*;

class ArrayListDemo {
  public static void main(String args[]) {
    // Create an array list.
    ArrayList<String> al = new ArrayList<String>();

    System.out.println("Initial size of al: " +
                       al.size());

    // Add elements to the array list.
    al.add("C");
    al.add("A");
    al.add("E");
    al.add("B");
    al.add("D");
    al.add("F");
    al.add(1, "A2");

    System.out.println("Size of al after additions: " +
                       al.size());

    // Display the array list.
    System.out.println("Contents of al: " + al);

    // Remove elements from the array list.
    al.remove("F");
    al.remove(2);

    System.out.println("Size of al after deletions: " +
                       al.size());
    System.out.println("Contents of al: " + al);
  }
}
```

The output from this program is shown here:

```
Initial size of al: 0
Size of al after additions: 7
Contents of al: [C, A2, A, E, B, D, F]
Size of al after deletions: 5
Contents of al: [C, A2, E, B, D]
```

Obtaining an Array from an ArrayList

```
// Convert an ArrayList into an array.
import java.util.*;

class ArrayListToArray {
  public static void main(String args[]) {
    // Create an array list.
    ArrayList<Integer> al = new ArrayList<Integer>();

    // Add elements to the array list.
    al.add(1);
    al.add(2);
    al.add(3);
    al.add(4);

    System.out.println("Contents of al: " + al);

    // Get the array.
    Integer ia[] = new Integer[al.size()];
    ia = al.toArray(ia);

    int sum = 0;

    // Sum the array.
    for(int i : ia) sum += i;

    System.out.println("Sum is: " + sum);
  }
}
```

The output from the program is shown here:

```
Contents of al: [1, 2, 3, 4]
Sum is: 10
```

The LinkedList Class

```
// Demonstrate LinkedList.
import java.util.*;

class LinkedListDemo {
  public static void main(String args[]) {
    // Create a linked list.
    LinkedList<String> ll = new LinkedList<String>();

    // Add elements to the linked list.
    ll.add("F");
    ll.add("B");
    ll.add("D");
    ll.add("E");
    ll.add("C");
    ll.addLast("Z");
    ll.addFirst("A");

    ll.add(1, "A2");

    System.out.println("Original contents of ll: " + ll);

    // Remove elements from the linked list.
    ll.remove("F");
    ll.remove(2);

    System.out.println("Contents of ll after deletion: "
                       + ll);

    // Remove first and last elements.
    ll.removeFirst();
    ll.removeLast();

    System.out.println("ll after deleting first and last: "
                       + ll);

    // Get and set a value.
    String val = ll.get(2);
    ll.set(2, val + " Changed");

    System.out.println("ll after change: " + ll);
  }
}
```

The output from this program is shown here:

```
Original contents of ll: [A, A2, F, B, D, E, C, Z]
Contents of ll after deletion: [A, A2, D, E, C, Z]
ll after deleting first and last: [A2, D, E, C]
ll after change: [A2, D, E Changed, C]
```

ACCESSING A COLLECTION VIA AN ITERATOR

- **Iterator** enables you to cycle through a collection, obtaining or removing elements.
- **ListIterator** extends **Iterator** to allow bidirectional traversal of a list, and the modification of elements.
- **Iterator** and **ListIterator** are generic interfaces which are declared as shown here:
- interface Iterator<E>
- interface ListIterator<E>

The Methods Defined by Iterator

Method	Description
boolean hasNext( )	Returns **true** if there are more elements. Otherwise, returns **false**.
E next( )	Returns the next element. Throws **NoSuchElementException** if there is not a next element.
void remove( )	Removes the current element. Throws **IllegalStateException** if an attempt is made to call **remove( )** that is not preceded by a call to **next( )**.

The Methods Defined by ListIterator

Method	Description
void add(E *obj*)	Inserts *obj* into the list in front of the element that will be returned by the next call to **next( )**.
boolean hasNext( )	Returns **true** if there is a next element. Otherwise, returns **false**.
boolean hasPrevious( )	Returns **true** if there is a previous element. Otherwise, returns **false**.
E next( )	Returns the next element. A **NoSuchElementException** is thrown if there is not a next element.
int nextIndex( )	Returns the index of the next element. If there is not a next element, returns the size of the list.
E previous( )	Returns the previous element. A **NoSuchElementException** is thrown if there is not a previous element.
int previousIndex( )	Returns the index of the previous element. If there is not a previous element, returns –1.
void remove( )	Removes the current element from the list. An **IllegalStateException** is thrown if **remove( )** is called before **next( )** or **previous( )** is invoked.
void set(E *obj*)	Assigns *obj* to the current element. This is the element last returned by a call to either **next( )** or **previous( )**.

Using an Iterator

- In general, to use an iterator to cycle through the contents of a collection, follow these steps:

1. Obtain an iterator to the start of the collection by calling the collection's **iterator( )** method.

2. Set up a loop that makes a call to **hasNext( )**. Have the loop iterate as long as **hasNext( )** returns **true**.

3. Within the loop, obtain each element by calling **next( )**.

Using an Iterator

```
// Demonstrate iterators.
import java.util.*;

class IteratorDemo {
  public static void main(String args[]) {
    // Create an array list.
    ArrayList<String> al = new ArrayList<String>();

    // Add elements to the array list.
    al.add("C");
    al.add("A");
    al.add("E");
    al.add("B");
    al.add("D");
    al.add("F");

    // Use iterator to display contents of al.
    System.out.print("Original contents of al: ");
    Iterator<String> itr = al.iterator();
    while(itr.hasNext()) {
      String element = itr.next();
      System.out.print(element + " ");
    }
    System.out.println();

    // Modify objects being iterated.
    ListIterator<String> litr = al.listIterator();
    while(litr.hasNext()) {
      String element = litr.next();
      litr.set(element + "+");
    }

    System.out.print("Modified contents of al: ");
    itr = al.iterator();
    while(itr.hasNext()) {
```

Using an Iterator

```
      String element = itr.next();
      System.out.print(element + " ");
    }
    System.out.println();

    // Now, display the list backwards.
    System.out.print("Modified list backwards: ");
    while(litr.hasPrevious()) {
      String element = litr.previous();
      System.out.print(element + " ");
    }
    System.out.println();
  }
}
```

The output is shown here:

```
Original contents of al: C A E B D F
Modified contents of al: C+ A+ E+ B+ D+ F+
Modified list backwards: F+ D+ B+ E+ A+ C+
```

The For-Each Alternative to Iterators

```
// Use the for-each for loop to cycle through a collection.
import java.util.*;

class ForEachDemo {
  public static void main(String args[]) {
    // Create an array list for integers.
    ArrayList<Integer> vals = new ArrayList<Integer>();

    // Add values to the array list.
    vals.add(1);
    vals.add(2);
    vals.add(3);
    vals.add(4);
    vals.add(5);

    // Use for loop to display the values.
    System.out.print("Original contents of vals: ");
    for(int v : vals)
      System.out.print(v + " ");

       System.out.println();

       // Now, sum the values by using a for loop.
       int sum = 0;
       for(int v : vals)
         sum += v;

       System.out.println("Sum of values: " + sum);
     }
   }
```

The output from the program is shown here:

```
Original contents of vals: 1 2 3 4 5
Sum of values: 15
```

Multithreading

Differences between multi-threading and multitasking

Multitasking:

- The operating system's ability to have more than one program working at the same time is known as multitasking.
- For example, you can print while editing or downloading your email.

Multithreading:

- Multithreaded programs extend the idea of multitasking by taking it one level lower: Individual programs will appear to do multiple tasks at the same time.
- Each task is usually called a thread, which is short for thread of control.
- Programs that can run more than one thread at once are said to be multithreaded.

Difference between multiple processes and multiple threads

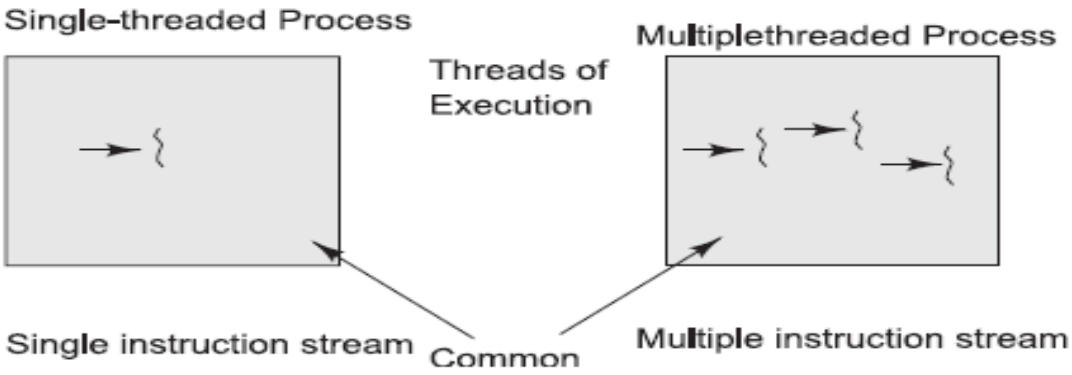
- The essential difference is that while each process has a complete set of its own variables, threads share the same data.
- However, shared variables make communication between threads more efficient and easier to program than inter-process communication.
- Moreover, on some operating systems, threads are more "lightweight" than processes—it takes less overhead to create and destroy individual threads than it does to launch new processes.
- Multithreading is extremely useful in practice.

For example,

- A browser should be able to simultaneously download multiple images.
- A web server needs to be able to serve concurrent requests.
- Graphical user interface (GUI) programs have a separate thread for gathering user interface events from the host operating environment.

DEFINING THREADS

- A process is a program in execution.
- A process may be divided into a number of independent units known as threads.
- A thread is a dispatchable unit of work.
- Threads are light-weight processes within a process.
- A process is a collection of one or more threads and associated system resources.
- A process may have a number of threads in it. A thread may be assumed as a subset of a process.



DEFINING THREADS

- If two applications are run on a computer (MS Word, MS Access), two processes are created.
- Multitasking of two or more processes is known as process-based multitasking.
- Multitasking of two or more threads is known as thread-based multitasking.
- The concept of multithreading in a programming language refers to thread-based multitasking.
- Process-based multitasking is totally controlled by the operating system.
- But thread-based multitasking can be controlled by the programmer to some extent in a program.

Context switching

- The concept of context switching is integral to threading.
- A hardware timer is used by the processor to determine the end of the time slice for each thread.
- The timer signals at the end of the time slice and in turn the processor saves all information required for the current thread onto a stack.
- Then the processor moves this information from the stack into a predefined data structure called a context structure.
- When the processor wants to switch back to a previously executing thread, it transfers all the information from the context structure associated with the thread to the stack. This entire procedure is known as context switching

DEFINING THREADS

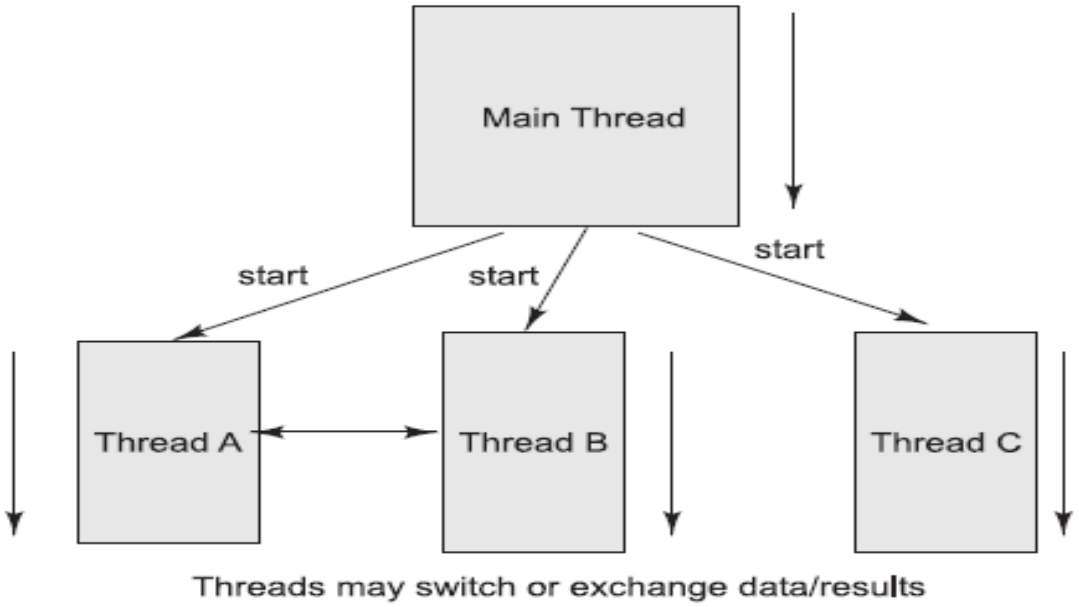
- Java supports thread-based multitasking .

The advantages of thread-based multitasking as compared to process-based multitasking are given below:

- Threads share the same address space.
- Context-switching between threads is normally inexpensive.
- Communication between threads is normally inexpensive.

THREADS IN JAVA:

- A single process might contain multiple threads.
- All threads within a process share the same state and same memory space, and can communicate with each other directly, because they share the same variables.



Creating Threads

- Threads are objects in the Java language. They can be created by two ways.

1. Create a class that extends the standard Thread class.
2. Create a class that implements the standard Runnable interface.

- That is, a thread can be defined by extending the java.lang.Thread class or by implementing the java.lang.Runnable interface.
- The run() method should be overridden and should contain the code that will be executed by the new thread.
- This method must be public with a void return type and should not take any arguments.

1. Extending the Thread Class

1. Create a class by extending the Thread class and override the run() method:

```
class MyThread extends Thread
{
    public void run()
    {
        // thread body of execution
    }
}
```

2. Create a thread object:

```
MyThread thr1 = new MyThread();
```

3. Start Execution of created thread:

```
thr1.start();
```

Example:

```
public class MyThread extends Thread
{
public void run()
   {
System.out.println("The thread is running");
   }
}
public class Example
{
public static void main(String[] args)
   {
MyThread t=new MyThread();
t.start();
}
}
```

Output:

The thread is running

2. Implementing the Runnable Interface

1. Create a class that implements the interface Runnable and override run() method:

```
class MyThread implements Runnable
{
…
public void run()
 {
// thread body of execution
}
}
```

2. Creating Object:

```
MyThread myObject = new MyThread();
```

3. Creating Thread Object:

```
Thread thr1 = new Thread(myObject);
```

4. Start Execution:

```
thr1.start();
```

Example:

```
public class MyThread implements Runnable
{
public void run()
   {
System.out.println("The thread is running");
   }
}
public class Example
{
public static void main(String[] args)
   {
            MyThread t=new MyThread();
  Thread tt=new Thread(t);
tt.start();
 }
}
```

Output:

The thread is running

Methods

java.lang.Thread

1. Thread(Runnable target)

constructs a new thread that calls the run() method of the specified target.

2. void start()

Starts this thread, causing the run() method to be called. This method will return immediately. The new thread runs concurrently.

3. void run()

calls the run method of the associated Runnable.

java.lang.Runnable

1. void run()

must be overriden and supplied with instructions for the task that you want to have executed.

Thread Life Cycle

Threads can be in one of six states:

- New
- Runnable
- Blocked
- Waiting
- Timed waiting
- Terminated

Thread Life Cycle

1. New Threads

- When you create a thread with the new operator—for example, new Thread(r)—the thread is not yet running.
- This means that it is in the new state.
- When a thread is in the new state, the program has not started executing code inside of it.

Thread Life Cycle

2. Runnable Threads

- Once you invoke the start method, the thread is in the runnable state. A runnable thread may or may not actually be running. It is up to the operating system to give the thread time to run.
- Once a thread is running, it doesn't necessarily keep running. In fact, it is desirable that running threads occasionally pause so that other threads have a chance to run. The details of thread scheduling depend on the services that the operating system provides.

contd..

Thread Life Cycle

2. Runnable Threads

- Preemptive scheduling systems give each runnable thread a slice of time to perform its task.
- When that slice of time is exhausted, the operating system preempts the thread and gives another thread an opportunity to work.
- When selecting the next thread, the operating system takes into account the thread priorities. All modern desktop and server operating systems use preemptive scheduling. However, small devices such as cell phones may use cooperative scheduling. In such a device, a thread loses control only when it calls the yield method, or it is blocked or waiting.
- On a machine with multiple processors, each processor can run a thread, and you can have multiple threads run in parallel. Of course, if there are more threads than processors, the scheduler still has to do time-slicing.
- Always keep in mind that a runnable thread may or may not be running at any given time.

Thread Life Cycle

3. Blocked and Waiting Threads

- When a thread is blocked or waiting, it is temporarily inactive. It doesn't execute any code and consumes minimal resources. It is up to the thread scheduler to reactivate it. The details depend on how the inactive state was reached.
- When the thread tries to acquire an intrinsic object lock (but not a Lock in the java.util.concurrent library) that is currently held by another thread, it becomes blocked. The thread becomes unblocked when all other threads have relinquished the lock and the thread scheduler has allowed this thread to hold it.

contd..

Thread Life Cycle

3. Blocked and Waiting Threads

- When the thread waits for another thread to notify the scheduler of a condition, it enters the waiting state. This happens by calling the Object.wait or Thread.join method, or by waiting for a Lock or Condition in the java.util.concurrent library. In practice, the difference between the blocked and waiting state is not significant.
- Several methods have a timeout parameter. Calling them causes the thread to enter the timed waiting state. This state persists either until the timeout expires or the appropriate notification has been received. Methods with timeout include Thread.sleep and the timed versions of Object.wait, Thread.join, Lock.tryLock, and Condition.await.

Thread Life Cycle

4. Terminated Threads

- A thread is terminated for one of two reasons:
  - It dies a natural death because the run method exits normally.
  - It dies abruptly because an uncaught exception terminates the run method.
- In particular, you can kill a thread by invoking its stop method. That method throws a ThreadDeath error object that kills the thread.

java.lang.Thread 1.0

- void join()
  - waits for the specified thread to terminate.
- void join(long millis)
  - waits for the specified thread to die or for the specified number of milliseconds to pass.

Thread Life Cycle

4. Terminated Threads (contd..)

- Thread.StategetState() 5.0

gets the state of this thread; one of NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, or TERMINATED.

- void stop()

  stops the thread. This method is deprecated.

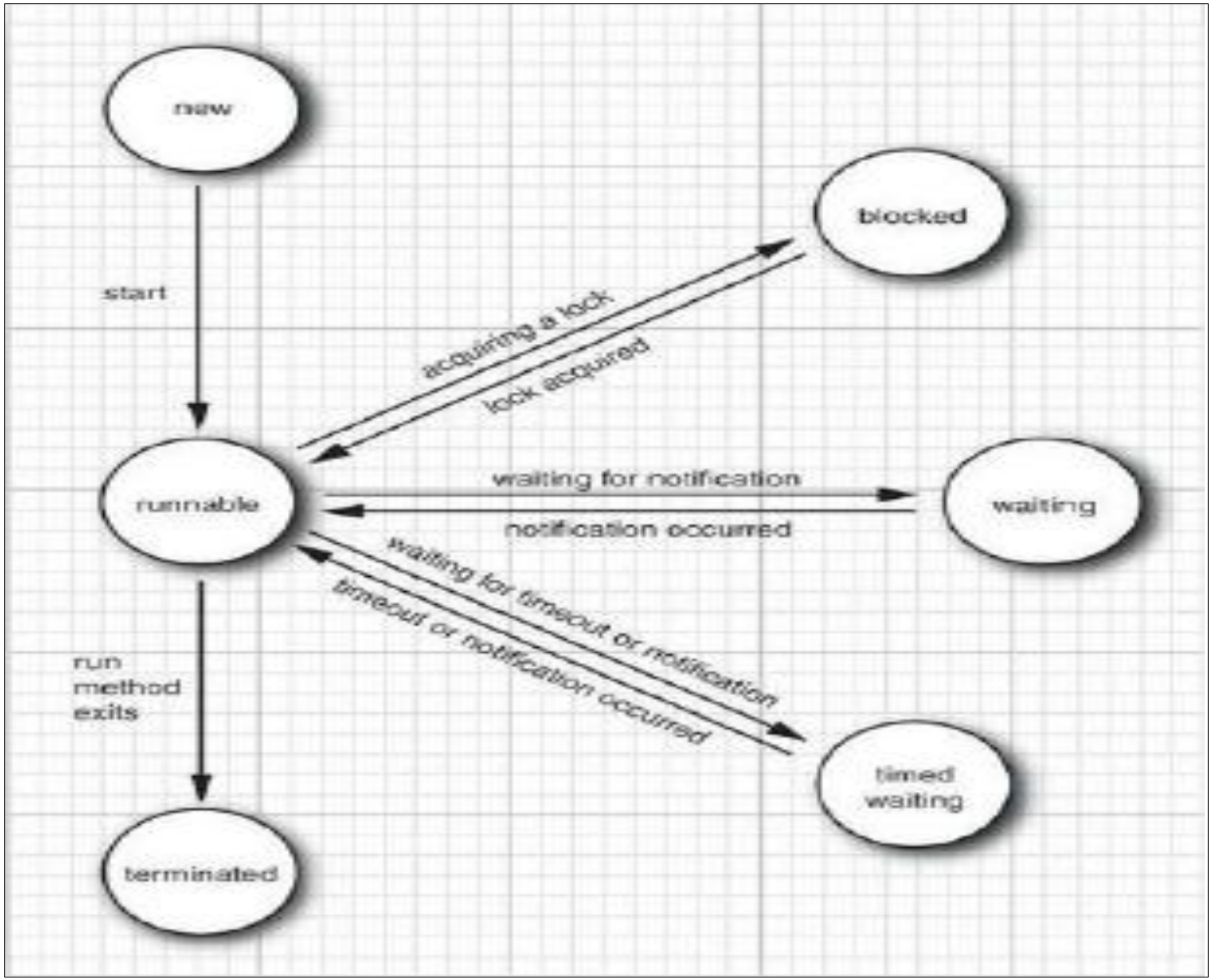
- void suspend()

suspends this thread's execution. This method is deprecated.

- void resume()

  resumes this thread. This method is only valid after suspend() has been invoked. This method is deprecated.

Thread Life Cycle



Thread Properties

- thread priorities
- daemon threads
- thread groups
- handlers for uncaught exceptions

1. Thread Priorities

- In the Java programming language, every thread has a priority.
- By default, a thread inherits the priority of the thread that constructed it.
- You can increase or decrease the priority of any thread with the setPriority method.

java.lang.Thread

- void setPriority(int newPriority)

sets the priority of this thread. The priority must be between Thread.MIN_PRIORITY andThread.MAX_PRIORITY. Use Thread.NORM_PRIORITY for normal priority.

- static int MIN_PRIORITY

  is the minimum priority that a Thread can have. The minimum priority value is 1.

- static int NORM_PRIORITY

  is the default priority of a Thread. The default priority is 5.

- static int MAX_PRIORITY

  is the maximum priority that a Thread can have. The maximum priority value is 10.

1. Thread Priorities (CONTD…)

- static void yield()

  causes the currently executing thread to yield. If there are other runnable threads with apriority at least as high as the priority of this thread, they will be scheduled next. Note that this is a static method
- Whenever the thread scheduler has a chance to pick a new thread, it prefers threads with higher priority.
- However, thread priorities are highly system-dependent. When the virtual machine relies on the thread implementation of the host platform, the Java thread priorities are mapped to the priority levels of the host platform, which may have more or fewer thread priority levels.
- For example, Windows has seven priority levels. Some of the Java priorities will map to the same operating system level.
- In the Oracle JVM for Linux, thread priorities are ignored altogether—all threads have the same priority.

2. Daemon Threads

- Daemon thread is a low priority thread that runs in background to perform tasks such as garbage collection.
- You can turn a thread into a daemon thread by calling

  t.setDaemon(true);
- There is nothing demonic about such a thread.
- A daemon is simply a thread that has no other role in life than to serve others.

Examples are

- timer threads that send regular “timer ticks” to other threads
- threads that clean up stale (out of date) cache entries.

Properties OF Daemon thread

- They can not prevent the JVM from exiting when all the user threads finish their execution.
- JVM terminates itself when all user threads finish their execution
- If JVM finds running daemon thread, it terminates the thread and after that shutdown itself. JVM does not care whether Daemon thread is running or not.
- It is an utmost low priority thread.
- Daemon threads are sometimes mistakenly used by beginners who don't want to think about shutdown actions. However, this can be dangerous. A daemon thread should never access a persistent resource such as a file or database since it can terminate at any time, even in the middle of an operation.

java.lang.Thread

- void setDaemon(boolean isDaemon)
- marks this thread as a daemon thread or a user thread. This method must be called before the thread is started.

3. Handlers for Uncaught Exceptions

- The run method of a thread cannot throw any checked exceptions, but it can be terminated by an unchecked exception. In that case, the thread dies.
- However, there is no catch clause to which the exception can be propagated. Instead, just before the thread dies, the exception is passed to a handler for uncaught exceptions.
- The handler must belong to a class that implements the Thread.UncaughtExceptionHandler interface. That interface has a single method,
  - void uncaughtException(Thread t, Throwable e)
- You can install a handler into any thread with the setUncaughtExceptionHandler method. You can also install a default handler for all threads with the static method setDefaultUncaughtExceptionHandler of the Thread class.
- A replacement handler might use the logging API to send reports of uncaught exceptions into a log file. If you don't install a default handler, the default handler is null. However, if you don't install a handler for an individual thread, the handler is the thread's ThreadGroup object.

4. ThreadGroup in Java

- Java provides a convenient way to group multiple threads in a single object. In such way, we can suspend, resume or interrupt group of threads by a single method call.
- Java thread group is implemented by java.lang.ThreadGroup class.
- A ThreadGroup represents a set of threads. A thread group can also include the other thread group.
- The thread group creates a tree in which every thread group except the initial thread group has a parent.
- A thread is allowed to access information about its own thread group, but it cannot access the information about its thread group's parent thread group or any other thread groups.

Constructors of ThreadGroup class

There are only two constructors of ThreadGroup class.

1)ThreadGroup(String name) - creates a thread group with given name.

2)ThreadGroup(ThreadGroup parent, String name) - creates a thread group with given parent group and name.

Methods of ThreadGroup class

1) void checkAccess() - This method determines if the currently running thread has permission to modify the thread group.

2)int activeCount() -This method returns an estimate of the number of active threads in the thread group and its subgroups.

3) int activeGroupCount()-This method returns an estimate of the number of active groups in the thread group and its subgroups.

4) void destroy() - This method destroys the thread group and all of its subgroups.

5) int enumerate(Thread[] list) - This method copies into the specified array every active thread in the thread group and its subgroups.

Methods of ThreadGroup class

6) int getMaxPriority() - This method returns the maximum priority of the thread group.
7) String getName() - This method returns the name of the thread group.
8) ThreadGroupgetParent() - This method returns the parent of the thread group.
9) void interrupt() - This method interrupts all threads in the thread group.
10) Boolean isDaemon() - This method tests if the thread group is a daemon thread group.
11) void setDaemon(boolean daemon) - This method changes the daemon status of the thread group.
12) Boolean isDestroyed() - This method tests if this thread group has been destroyed.

Methods of ThreadGroup class

13) void list() - This method prints information about the thread group to the standard output.

14) Boolean parentOf(ThreadGroup g) - This method tests if the thread group is either the thread group argument or one of its ancestor thread groups.

15) void suspend() - This method is used to suspend all threads in the thread group.

16) void resume() - This method is used to resume all threads in the thread group which was suspended using suspend() method.

17) void setMaxPriority(int pri) - This method sets the maximum priority of the group.

18) void stop() - This method is used to stop all threads in the thread group.

19) String toString() - This method returns a string representation of the Thread group.

Example:

```
class ThreadGroupDemo implements Runnable{
public void run() {
System.out.println(Thread.currentThread().getName());
   }
}
class Test
{
public static void main(String[] args)
 {
ThreadGroupDemo runnable = new ThreadGroupDemo();
ThreadGroup tg1 = new ThreadGroup("Parent ThreadGroup");
         Thread t1 = new Thread(tg1, runnable,"one");
t1.start();
        Thread t2 = new Thread(tg1, runnable,"two");
t2.start();
        Thread t3 = new Thread(tg1, runnable,"three");
t3.start();
System.out.println("Thread Group Name: "+tg1.getName());
tg1.list();
     }
  }
```

Output:
one
two
three
Thread Group Name: Parent ThreadGroup
java.lang.ThreadGroup[name=Parent ThreadGroup,maxpri=10]
Thread[one,5,Parent ThreadGroup]
Thread[two,5,Parent ThreadGroup]
Thread[three,5,Parent ThreadGroup]

Interthread Communication

- Multithreading replaces event loop programming by dividing your tasks into discrete, logical units.
- Threads also provide a secondary benefit: they do away with polling.
- **Polling** is usually implemented by a loop that is used to check some condition repeatedly.
- Once the condition is true, appropriate action is taken. This wastes CPU time.
- To avoid polling, Java includes an elegant interprocess communication mechanism via the **wait( ), notify( ), and notifyAll( )** methods.
- These methods are implemented as final methods in Object, so all classes have them.

Interthread Communication

- All three methods can be called only from within a synchronized context.
- The rules for using these methods are actually quite simple:

• wait( ) tells the calling thread to give up the monitor and go to sleep until some other thread enters the same monitor and calls notify( ).

• notify( ) wakes up a thread that called wait( ) on the same object.

• notifyAll( ) wakes up all the threads that called wait( ) on the same object. One of the threads will be granted access.

These methods are declared within Object, as shown here:

- final void wait( ) throws InterruptedException
- final void notify( )
- final void notifyAll( )

Interthread Communication

- Additional forms of wait( ) exist that allow you to specify a period of time to wait.
- Although wait( ) normally waits until notify( ) or notifyAll( ) is called, there is a possibility that in very rare cases the waiting thread could be awakened due to a spurious wakeup.
- In this case, a waiting thread resumes without notify( ) or notifyAll( ) having been called. (In essence, the thread resumes for no apparent reason.)
- Because of this remote possibility, Sun recommends that calls to wait( ) should take place within a loop that checks the condition on which the thread is waiting.

Producer/ consumer problem

- Producer thread produces an item and consumer thread consumes that item.
- Producer has to wait when the queue is full and the consumer has to wait until producer produce an item in the queue. So we need to synchronize the Queue.
- Program consists of four classes:

➢ Q, the queue that you're trying to synchronize;

➢ Producer, the threaded object that is producing queue entries;

➢ Consumer, the threaded object that is consuming queue entries;

➢ PC, the tiny class that creates the single Q, Producer, and Consumer.

```
// An incorrect implementation of a
producer and consumer.
class Q
{
        int n;
        synchronized int get()
        {

        System.out.println("Got: " + n);
                return n;
        }
        synchronized void put(int n)
        {
                this.n = n;

        System.out.println("Put: " + n);
        }
}
```

```
class Producer implements Runnable
{
        Q q;
        Producer(Q q)
        {
        this.q = q;
        new Thread(this,
                "Producer").start();
        }
        public void run()
        {
                int i = 0;
                while(true)
                {
                        q.put(i++);
                }
        }
}
```

```
class Consumer implements Runnable
{
        Q q;
        Consumer(Q q)
        {
        this.q = q;
        new Thread(this,
        "Consumer").start();
        }
        public void run()
        {
                while(true)
                {
                        q.get();
                }
        }
}
```

```
class PC
{
public static void main(String args[])
{
        Q q = new Q();
        new Producer(q);
        new Consumer(q);
        System.out.println("Press
Control-C to stop.");
}
}
```

Although the put( ) and get( ) methods on Q are synchronized, nothing stops the producer from overrunning the consumer, nor will anything stop the consumer from consuming the same queue value twice. Thus, you get the erroneous output shown here
(the exact output will vary with processor speed and task load):

```
Put: 1
Got: 1
Got: 1
Got: 1
Got: 1
Got: 1
Put: 2
Put: 3
Put: 4
Put: 5
Put: 6
Put: 7
Got: 7
```

As you can see, after the producer put 1, the consumer started and got the same 1 five times in a row. Then, the producer resumed and produced 2 through 7 without letting the consumer have a chance to consume them.

The proper way to write this program in Java is to use wait( ) and notify( ) to signal in both directions.

```
// A correct implementation of a producer and consumer.
class Q
{
        int n;
        boolean valueSet = false;
        synchronized int get()
        {
                while(!valueSet)
                try
                {
                        wait();
                }
        catch(InterruptedException e)
        {
                System.out.println("InterruptedException caught");
        }
        System.out.println("Got: " + n);
        valueSet = false;
        notify();
        return n;
        }
```

```
synchronized void put(int n)
{
        while(valueSet)
        try
        {
                wait();
        }
        catch(InterruptedException e)
        {
                System.out.println("InterruptedException caught");
        }
        this.n = n;
        valueSet = true;
        System.out.println("Put: " + n);
        notify();
}
}
```

```
class Producer implements Runnable
{
        Q q;
        Producer(Q q)
        {
                this.q = q;
                new Thread(this, "Producer").start();
        }
        public void run()
        {
                int i = 0;
                while(true)
                {
                        q.put(i++);
                }
        }
}
```

```
class Consumer implements Runnable
{
        Q q;
        Consumer(Q q)
        {
                this.q = q;
                new Thread(this, "Consumer").start();
        }
        public void run()
        {
                while(true)
                {
                        q.get();
                }
        }
}
```

```
class PCFixed
{
public static void main(String args[])
{
        Q q = new Q();
        new Producer(q);
        new Consumer(q);
        System.out.println("Press Control-C to stop.");
}
}
```

```
Put: 1
Got: 1
Put: 2
Got: 2
Put: 3
Got: 3
Put: 4
Got: 4
Put: 5
Got: 5
```

Inside get( ), wait( ) is called. This causes its execution to suspend until the Producer notifies you that some data is ready. When this happens, execution inside get( ) resumes. After the data has been obtained, get( ) calls notify( ). This tells Producer that it is okay to put more data in the queue. Inside put( ), wait( ) suspends execution until the Consumer has removed the item from the queue. When execution resumes, the next item of data is put in the queue, and notify( ) is called. This tells the Consumer that it should now remove it.

Generic programming

Generic programming

- Generic programming means writing code that can be reused for objects of many different types.
- The term generics mean parameterized types.
- Parameterized types are important because they enable you to create classes, interfaces, and methods in which the type of data upon which they operate is specified as a parameter.
- Using generics, it is possible to create a single class that automatically works with different types of data**.**
- **A class, interface, or method that operates on a parameterized type is called generic, as in generic class or generic method.**

Generic class:

A generic class is a class with one or more type variables.

The General Form of a Generic Class

```
class class-name<type-param-list>
{ // ...
}
```

Here is the syntax for declaring a reference to a generic class:

class-name<type-arg-list> var-name = new class-name<type-arg-list>(cons-arg-list);

for example, consider the class,

```
class Gen<T>
{
}
```

T is the name of a type parameter. This name is used as a placeholder for the actual type that will be passed to Gen when an object is created.
Gen is a generic class, which is also called a parameterized type.

```
public class Test<T>
{
    T a;
   Test (T x)
   {       a=x;
   }
   void print()
   {      System.out.println("a="+a);
   }
}
public class JavaApplication
{
    public static void main(String[] args)
    {      Test <Integer> n1=new Test <Integer>(2);
             n1.print();
            Test <Character> n2=new Test <Character>('v');
             n2.print();
             Test <String> n3=new Test <String>("hello");
            n3.print();
   } }
```

```
public class Test<T>
{    T a;
    Test (T x)
   {       a=x;
    }
   T getA()
   {       return a;
   }
}
public class JavaApplication5 {
         public static void main(String[] args) {
        Test <Integer> n1=new Test <Integer>(2);
        System.out.println("a="+n1.getA());
        Test <Character> n2=new Test <Character>('v');
        System.out.println("a="+n2.getA());
        Test <String> n3=new Test <String>("hello");
         System.out.println("a="+n3.getA());
    }
}
```

Output:

a=2

a=v

a=hello

- Java compiler does not actually create different versions of any other generic class.
- Instead, the compiler removes all generic type information, substituting the necessary casts, to make your code behave as if a specific version of Generic class were created.
- Thus, there is really only one version of Generic class that actually exists in your program.
- **The process of removing generic type information is called erasure.**
- ***Type checking is one of the main benefits of generics because it ensures type safety.***

Generics Work Only with Objects

- When declaring an instance of a generic type, the type argument passed to the type parameter must be a class type.
- We cannot use a primitive type, such as int or char. For example, with Gen class, it is possible to pass any class type to T, but you cannot pass a primitive type to a type parameter. Therefore, the following declaration is illegal:
- Gen<int> strOb = new Gen<int>(53); // Error, can't use primitive type
- We can use the **type wrappers** to encapsulate a primitive type.

Generic Types Differ Based on Their Type Arguments

A reference of one specific version of a generic type is not type compatible with another version of the same generic type. For example,

```
class Gen<T>
{
        …
}
Gen<Integer> iOb = new Gen<Integer>(53);
Gen<String> strOb = new Gen<String>(“hello”);
iOb= strOb;//leads to compiler error
```

Even though both iOb and strOb are of type Gen<T>, they are references to different types because their type parameters differ. This is part of the way that generics add type safety and prevent errors.

A Generic Class with Two Type Parameters

You can declare more than one type parameter in a generic type. To specify two or more type parameters, simply use a comma-separated list.

General form:

```
public class classname <T1,T2>
{
        ....
}
```

For example consider the class,

```
class TwoGen<T, V>
{
}
```

It specifies two type parameters: T and V, separated by a comma. Because it has two type parameters, two type arguments must be passed to TwoGen when an object is created.

TwoGen<**Integer, String**> tgObj = new TwoGen<Integer, String>(**88, "Generics**");

```
public class Test<T1,T2>
{
   T1 a;
   T2 b;
  Test (T1 x,T2 y)
  {
     a=x;
     b=y;
  }
  void print()
  {
     System.out.println("a="+a);
     System.out.println("b="+b);
  }
 }
public class JavaApplication {
   public static void main(String[] args)
{
  Test<Integer,Integer> n1=new Test
            <Integer,Integer>(2,3);
      n1.print();

 Test<Character,Integer> n2=new
Test<Character,Integer> ('v',5);
      n2.print();

 Test<String,Character> n3=new
Test<String,Character> ("hello",'g');
      n3.print();
  }
}
```

Output:

```
a=2
b=3
a=v
b=5
a=hello
b=g
```

Bounded Types

- The type parameters could be replaced by any class type.
- sometimes it is useful to **limit the types** that can be passed to a type parameter.
- Java provides bounded types.
- When specifying a type parameter, you can create an **upper bound that declares the superclass** from which all type arguments must be derived.
- This is accomplished through the use of an **extends clause** when specifying the type parameter, as shown here:

  <T extends superclass>

- This specifies that T can only be replaced by superclass, or subclasses of superclass. Thus, superclass defines an inclusive, upper limit.

Example 1:

```
public class Test<T extends Number>
{
   T[] a;
   Test(T[] x)
   {       a=x;
   }
   double average()
   {
      double avg=0.0;
  for(int i=0;i<a.length;i++)
      {
         avg+=a[i].doubleValue();
      }
      return avg/a.length;
}
public class Test1
{
 public static void main(String[] args)
{
  Integer x[]={1,2,3,4,5};
  Test<Integer> t=new Test<Integer>(x);

S.o.p(t.average());

Double y[]={1.1,2.2,3.3,4.4,5.5};
Test<Double > t1=new Test<Double >(y);

S.o.p(t1.average());
    }
}
```

Output:
3.0
3.3

Example 2:

```
public class Test<T extends Number>
{
   T a,b;
   Test(T x,T y)
   {
      a=x;
      b=y;
   }
  void  print()
   {          if(a instanceof Integer)
              {
              System.out.println("Integer");
              int x=a.intValue();
              int y=b.intValue();
              System.out.println(x+y);
              }
              else if(a instanceof Float)
              {
              System.out.println("Float");
              float x=a.floatValue();
              float y=b.floatValue();
              System.out.println(x+y);
              }
              else if(a instanceof Double)
              {
              System.out.println("Double");
              double x=a.doubleValue();
              double y=b.doubleValue();
              System.out.println(x+y);
              }
    }
}
public class Test1
 {
    public static void main(String[] args)
 {
        Test<Integer> t1=new
Test<Integer>(2,3);
         t1.print();
         Test<Double> t2=new
Test<Double>(2.5,3.5);
         t2.print();
         Test<Float> t3=new
Test<Float>(2.4f,3.3f);
         t3.print();
}
}
```

Output:

```
Integer
5
Double
6.0
Float
5.7
```

- In addition to using a class type as a bound, we can also use an interface type.
- We can specify multiple interfaces as bounds.
- A bound can include both a class type and one or more interfaces. In this case, the class type must be specified first.
- When a bound includes an interface type, only type arguments that implement that interface are legal.
- When specifying a bound that has a class and an interface, or multiple interfaces, use the & operator to connect them.
- For example,

```
class Gen<T extends MyClass & MyInterface> { // ...}
```

```
public class Test<T extends Comparable>
{
   T[] a;
   Test(T [] x)
   {
      a=x;
   }
  void  max()
   {
     T temp=a[0];
  for(int i=0;i<a.length;i++)
           {
    if((temp.compareTo(a[i]))<0)
    {
           temp=a[i];
    }
  }
S.o.p("Max elemnt="+temp);
  }
}
public class Test1
 {
   public static void main(String[] args)
 {
  Integer[] x={1,2,3,4,5};
  Double[] y={1.1,2.2,3.3,4.4,5.5};
  Test<Integer> t1=new Test<Integer>(x);
  t1.max();
  Test<Double> t2=new Test<Double>(y);
  t2.max();
}
}
```

Output:

Max elemnt=5

Max elemnt=5.5

Creating a Generic Method:

- Generic methods are methods that introduce their own type parameters. This is similar to declaring a generic type, but the type parameter's scope is limited to the method where it is declared. Static and non-static generic methods are allowed, as well as generic class constructors.

Generic Form:

<type-param-list> ret-type meth-name(param-list) { // ...}

- In all cases, type-param-list is a comma-separated list of type parameters. Notice that for a generic method, the type parameter list precedes the return type.
- Note that the type variables are **inserted after the modifiers (public static, in our case) and before the return type.**
- You can define generic methods both inside ordinary classes and inside generic classes. When you call a generic method, you can place the actual types, enclosed in angle brackets, before the method name. you can omit the type parameter from the methodcall.
- The compiler has enough information to infer the method that you want. It matches the type of actual parameters that are passed and deduces the actual type of T.

Example 1:

```
public class Test
{
   <T> void read(T a)
   {
      System.out.println("element read="+a);
   }
}
public class JavaApplication5
{
   public static void main(String[] args) {
    Test t=new Test();
    t.read(2);
    t.read('c');
    t.read("hello");
   }
   }
```

Output:
element read=2
element read=c
element read=hello

Example 2:

```
public class Test
{   public <T extends Comparable> void sort(T[] a)
   {
      for(int i=0;i<a.length;i++)
      {
         for(int j=i+1;j<a.length;j++)
         {
            if(a[i].compareTo(a[j])>0)
            {        T temp=a[i];
                     a[i]=a[j];
                     a[j]=temp;
            }
         }
      }
      for(int i=0;i<a.length;i++)
         System.out.println(a[i]);
      } }
```

```
public class NewGeneric {
public static void main(String[] args)
{
        Integer x[]={7,2,6,4,5};
        Test t1=new Test();
         t1.sort(x);//t1.<Integer>sort(x);
        Double y[]={9.1,2.2,6.3,4.4,5.5};
        Test t2=new Test();
        t2.sort(y); //t2.<Double>sort(x);
        Character z[]={'d','z','t','a','c'};
        t2.sort(z); //t3.<Character>sort(x);
   }

}
```

Example 3:

```
public class Test
{
   public <T extends Comparable<T>> void max(T[] a)
   {
     T temp=a[0];
      for(int i=0;i<a.length;i++)
      {

                        if((temp.compareTo(a[i]))<0)
                        {
                                  temp=a[i];
                        }
      }
    System.out.println("maximum elemnt="+temp);
   }
}
```

```
public class JavaApplication {
    public static void main(String[] args) {
      Test t=new Test();
      Integer[] x={1,2,3,4,5};
      t.max(x);
      Double[] y={1.1,2.2,3.3,4.4,5.5};
      t.max(y);
    }
 }
```

Output:

maximum elemnt=5

maximum elemnt=5.5

Generic Methods in Generic Classes

We can also have generic methods inside generic classes.

```
public class Data <T extends Comparable>
{
   T a[],temp;
   Data(T x[])
   {
      a=x;
   }
   public <T extends Comparable> void max()
   {
      temp=a[0];
      for(int i=1;i<a.length;i++)
      {
         if(temp.compareTo(a[i])<0)
         {
            temp=a[i];
         }
      }
      System.out.println("Maximum element ="+temp);
      }   }
```

```
public class Test
{
    public static void main(String[] args) {
        Integer [] a={3,2,7,9,16};
      Double [] b={3.5,7.4,2.1,7.8,5.6};
      Data<Integer> d1=new Data<Integer>(a);
      d1.max();
       Data<Double> d2=new Data<Double>(b);
      d2.max();
   }
}
```

Output:

Maximum element =16

Maximum element =7.8

Generic Interface

Like generic classes, you can also define generic interfaces. The same syntax used to define generic classes is also used to define generic interfaces.

Example:

```
public interface shape<T> {
   public void area();
}
public class Rectangle implements shape {
   Integer l,b;
   Rectangle(Integer  x,Integer  y)
   {
      l=x;
      b=y;
   }
```

```
 public void area()
   {
      System.out.println("Area of Rectangle="+(l*b));
   }
}
public class Square implements shape
{
   Integer a;
   Square(Integer x)
   {
      a=x;
   }
   public void area()
   {
      System.out.println("Area of Square="+(a*a));
   }
}
```

```
public class Test {
    public static void main(String[] args) {
      shape <Integer> s;
      s=new Rectangle(2,3);
      s.area();
      s=new Square(2);
      s.area();
        }
  }
```

Output:

Area of Rectangle=6

Area of Square=4

RESTRICTIONS AND LIMITATIONS ON GENERICS

To use Java generics effectively, you must consider the following restrictions:

1. Cannot Instantiate Generic Types with Primitive Types
2. Cannot Create Instances of Type Parameters
3. Cannot Declare Static Fields Whose Types are Type Parameters
4. Cannot Use Casts or instanceof With Parameterized Types
5. Cannot Create Arrays of Parameterized Types
6. Cannot Create, Catch, or Throw Objects of Parameterized Types
7. Cannot Overload a Method Where the Formal Parameter Types of Each Overload Erase to the Same Raw Type

1. Cannot Instantiate Generic Types with Primitive Types

Consider the following parameterized type:

```
class Pair<K, V>
{
    private K key;
    private V value;
    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }
    // ...
}
```

1. Cannot Instantiate Generic Types with Primitive Types

- When creating a Pair object, you cannot substitute a primitive type for the type parameter K or V:

```
Pair<int, char> p = new Pair<>(8, 'a');  // compile-time error
```

- You can substitute only non-primitive types for the type parameters K and V:

```
Pair<Integer, Character> p = new Pair<>(8, 'a');
```

- Note that the Java compiler autoboxes 8 to Integer.valueOf(8) and 'a' to Character('a'):

- Pair<Integer, Character> p = new Pair<>(Integer.valueOf(8), new Character('a'));

2. Cannot Create Instances of Type Parameters

You cannot create an instance of a type parameter. For example, the following code causes a compile-time error:

```
public static <E> void append(List<E> list) {
   E elem = new E();  // compile-time error
   list.add(elem);
}
```

As a workaround, you can create an object of a type parameter through reflection:

```
public static <E> void append(List<E> list, Class<E> cls) throws Exception {
   E elem = cls.newInstance();   // OK
   list.add(elem);
}
```

You can invoke the append method as follows:

```
List<String> ls = new ArrayList<>();
append(ls, String.class);
```

3. Cannot Declare Static Fields Whose Types are Type Parameters

A class's static field is a class-level variable shared by all non-static objects of the class. Hence, static fields of type parameters are not allowed. Consider the following class:

```
public class MobileDevice<T> {
   private static T os;

   // ...
}
```

If static fields of type parameters were allowed, then the following code would be confused:

```
MobileDevice<Smartphone> phone = new MobileDevice<>();
MobileDevice<Pager> pager = new MobileDevice<>();
MobileDevice<TabletPC> pc = new MobileDevice<>();
```

Because the static field os is shared by phone, pager, and pc, what is the actual type of os? It cannot be Smartphone, Pager, and TabletPC at the same time. You cannot, therefore, create static fields of type parameters.

4. Cannot Use Casts or instanceof with Parameterized Types

Because the Java compiler erases all type parameters in generic code, you cannot verify which parameterized type for a generic type is being used at runtime:

```
public static <E> void rtti(List<E> list) {
   if (list instanceof ArrayList<Integer>) {  // compile-time error
      // ...
   }
}
```

The set of parameterized types passed to the rtti method is:

S = { ArrayList<Integer>, ArrayList<String> LinkedList<Character>, ... }

The runtime does not keep track of type parameters, so it cannot tell the difference between an ArrayList<Integer> and an ArrayList<String>. The most you can do is to use an unbounded wildcard to verify that the list is an ArrayList:

```
public static void rtti(List<?> list) {
   if (list instanceof ArrayList<?>) {  // OK; instanceof requires a reliable type
      // ...
   }
}
```

4. Cannot Use Casts or instanceof with Parameterized Types

Typically, you cannot cast to a parameterized type unless it is parameterized by unbounded wildcards. For example:

```
List<Integer> li = new ArrayList<>();
List<Number>  ln = (List<Number>) li;  // compile-time error
```

However, in some cases the compiler knows that a type parameter is always valid and allows the cast. For example:

```
List<String> l1 = ...;
ArrayList<String> l2 = (ArrayList<String>)l1;  // OK
```

5. Cannot Create Arrays of Parameterized Types

You cannot create arrays of parameterized types. For example, the following code does not compile:

```
List<Integer>[] arrayOfLists = new List<Integer>[2];  // compile-time error
```

The following code illustrates what happens when different types are inserted into an array:

```
Object[] strings = new String[2];
strings[0] = "hi";   // OK
strings[1] = 100;    // An ArrayStoreException is thrown.
```

If you try the same thing with a generic list, there would be a problem:

```
Object[] stringLists = new List<String>[];  // compiler error, but pretend it's allowed
stringLists[0] = new ArrayList<String>();   // OK
stringLists[1] = new ArrayList<Integer>();  // An ArrayStoreException should be thrown,
                                            // but the runtime can't detect it.
```

If arrays of parameterized lists were allowed, the previous code would fail to throw the desired ArrayStoreException.

6. Cannot Create, Catch, or Throw Objects of Parameterized Types

A generic class cannot extend the Throwable class directly or indirectly. For example, the following classes will not compile:

```
// Extends Throwable indirectly
class MathException<T> extends Exception { /* ... */ }    // compile-time error

// Extends Throwable directly
class QueueFullException<T> extends Throwable { /* ... */ // compile-time error
```

A method cannot catch an instance of a type parameter:

```
public static <T extends Exception, J> void execute(List<J> jobs) {
   try {
      for (J job : jobs)
         // ...
   } catch (T e) {   // compile-time error
      // ...
   }
}
```

6. Cannot Create, Catch, or Throw Objects of Parameterized Types

You can, however, use a type parameter in a throws clause:

```
class Parser<T extends Exception> {
    public void parse(File file) throws T {     // OK
        // ...
    }
}
```

7. Cannot Overload a Method Where the Formal Parameter Types of Each Overload Erase to the Same Raw Type

A class cannot have two overloaded methods that will have the same signature after type erasure.

```
public class Example {
   public void print(Set<String> strSet) { }
   public void print(Set<Integer> intSet) { }
}
```

The overloads would all share the same class file representation and will generate a compile-time error.